

Hochschule
Zittau/Görlitz
UNIVERSITY OF APPLIED SCIENCES

JavaScript Deepdive

Web Engineering 1 (319300)

Vorlesung 03

M.Sc. Christopher-Manuel Hilgner

Hochschule Zittau/Görlitz

christopher.hilgner@stud.hszg.de

2026-10-20

Agenda

- JavaScript
- JSON
- Client-seitige Validierung

JavaScript

Überblick

- Implementierung des ECMAScript-Standards
- JavaScript ist kein formal definierter Name
- ECMAScript ist die eigentliche Spezifikation
- Unterschiedliche Engines implementieren sie: V8, SpiderMonkey, JavaScriptCore
- Seit 2015 jährlicher Releasezyklus mit neuen Features (z.B.: `let/const`, Arrow Functions, `async/await`)

JavaScript

```
1    let num;
2    const cNum = 0;
3    const func = asnyc () => {
4
5    }
6    await func();
```

Interpretiert oder kompiliert?

- Häufige Vereinfachung *JavaScript ist interpretiert* stimmt nur bedingt
- Engines wie V8 nutzen JIT-Kompilierung (Just-In-Time)
  - Schnelle interpretation in Bytecode
  - Häufiger genutzter Code wird in Maschinencode übersetzt

Kurz: Es gibt keinen Kompilierschritt wie bei Java. Der Code wird direkt *ausgeführt*. *Ausgeführt* heißt hier aber mehr als reines Zeile-für-Zeile interpretieren.

Single-Threaded & der Call-Stack

- JavaScript hat genau einen Ausführungs-Thread
- Es kann immer nur eine Zeile ausgelesen und ausgeführt werden
- Modell dafür: der **Call Stack**
- Jeder Funktionsaufruf legt neuen Frame auf den Stack
- Bei `return` wird der Frame entfernt
- Endlose Rekursion führt zu `Maximum call stack size exceeded`

Konsequenz: Eine lang laufende, synchrone Operation blockiert die komplette Seite (keine Klicks, keine Animationen)

Folgefrage: Wie schafft es JavaScript auf eine Netzwerk Request zu warten ohne die Seite einzufrieren?

Event Loop, Task Queue und Browser-APIs

Wichtigster Punkt für JavaScript Verständnis

1. JavaScript kann weder Timer noch Netzwerk-Requests. Das übernehmen Browser-APIs (z.B.: bei Node.js ist es die C++-Runtime): `setTimeout`, `fetch`, DOM-Events
2. Nach Beendigung der async Operation wird Callback nicht sofort ausgeführt. Sie wird in Task Queue gelegt
3. Event Loop prüft durchgehend, ob Call-Stack leer ist. Falls ja wird die nächste Aufgabe aus der Queue auf den Stack geschoben

Merksatz: JavaScript ist single-threaded, aber die Browser-Umgebung drumherum ist es nicht. JS delegiert Wartezeiten nach außen und bekommt nur das fertige Ergebnis zurückgemeldet, sobald Platz im Stack ist.

Microtasks vor Macrotasks

- Promise-Callbacks (`.then`, `async/await`) werden immer vor `setTimeout`-Callbacks abgearbeitet

Dynamische und schwache Typisierung

Dynamisch: der Typ einer Variable wird zur Laufzeit bestimmt und kann sich ändern

Dynamische und schwache Typisierung

Dynamisch: der Typ einer Variable wird zur Laufzeit bestimmt und kann sich ändern

```javascript
let x = 5; x = "fünf"; //gültig
```

Dynamische und schwache Typisierung

Dynamisch: der Typ einer Variable wird zur Laufzeit bestimmt und kann sich ändern

```JavaScript
let x = 5; x = "fünf"; //gültig
```

Schwach typisiert: Operatoren erwingen implizite Typumwandlung zwischen unterschiedlichen Typen

Dynamische und schwache Typisierung

Dynamisch: der Typ einer Variable wird zur Laufzeit bestimmt und kann sich ändern

```JavaScript
let x = 5; x = "fünf"; //gültig
```

Schwach typisiert: Operatoren erwingen implizite Typumwandlung zwischen unterschiedlichen Typen

```JavaScript
"5" + 1 // "51"
"5" - 1 // 4
```

Prototypenbasierte Objektorientierung

- Anders als Java nutzt JavaScript **Prototypen**
  - Jedes Objekt hat interne Verknüpfung zu einem anderen Objekt
  - Anderes Objekt vererbt Eigenschaften
  - `[[Prototype]]`, zugreifbar über `__proto__` oder `Object.getPrototypeOf()`
- `class`-Syntax ist nur syntaktischer Zucker Prototypen
- Vererbung in JS ist Objekt-zu-Objekt nicht Klasse-zu-Klasse

Garbage Collection

- JS verwaltet Speicher automatisch über einen Garbage Collector (meist Mark-and-Sweep-Verfahren)
- Objekte, die von keiner erreichbaren Referenz mehr aus dem Programm aus erreichbar sind werden freigegeben
- Kein manuelles `malloc`/`free` wie in C
- Nicht erreichbare Closures, vergessene Event-Listener können trotzdem zu Memory Leaks führen

Execution Context & Hoisting

Vor Codeausführung: Engine erstellt für jede Funktion/jedes Skript einen Execution Context in zwei Phasen:

Execution Context & Hoisting

Vor Codeausführung: Engine erstellt für jede Funktion/jedes Skript einen Execution Context in zwei Phasen:

1. **Creation Phase**: Variablen- und Funktionsdeklarationen werden im Speicher registriert (Hoisting)
   - `var`-Variablen mit `undefined`
   - `function`-Deklarationen vollständig

Execution Context & Hoisting

Vor Codeausführung: Engine erstellt für jede Funktion/jedes Skript einen Execution Context in zwei Phasen:

1. **Creation Phase**: Variablen- und Funktionsdeklarationen werden im Speicher registriert (Hoisting)
   - `var`-Variablen mit `undefined`
   - `function`-Deklarationen vollständig
2. **Execution Phase**: Code wird Zeile für Zeile tatsächlich ausgeführt

Execution Context & Hoisting

Vor Codeausführung: Engine erstellt für jede Funktion/jedes Skript einen Execution Context in zwei Phasen:

1. **Creation Phase**: Variablen- und Funktionsdeklarationen werden im Speicher registriert (Hoisting)
   - `var`-Variablen mit `undefined`
   - `function`-Deklarationen vollständig
2. **Execution Phase**: Code wird Zeile für Zeile tatsächlich ausgeführt

Deshalb kann eine Funktion vor ihrer Definition aufgerufen werden. `const`/`let` Deklarationen würden `ReferenceError` werfen.

Designprinzipien Zusammenfassung

- **Funktionen sind First-Class Citizens**: Grundlage für Callbacks, Higher-Order-Funktionen, funktionale Patterns (`map`/`filter`/`reduce`)

Designprinzipien Zusammenfassung

- **Funktionen sind First-Class Citizens**: Grundlage für Callbacks, Higher-Order-Funktionen, funktionale Patterns (`map`/`filter`/`reduce`)
- **Alles, was kein Primitive ist, ist ein Objekt**: Arrays, Funktionen, sogar `null` (historisch fälschlich als „object“ typisiert)

Designprinzipien Zusammenfassung

- **Funktionen sind First-Class Citizens**: Grundlage für Callbacks, Higher-Order-Funktionen, funktionale Patterns (`map`/`filter`/`reduce`)
- **Alles, was kein Primitive ist, ist ein Objekt**: Arrays, Funktionen, sogar `null` (historisch fälschlich als „object“ typisiert)
- **Prototypenbasierte Vererbung statt Klassenhierarchie**: Sehr flexibel aber ungewohnt mit typischem OOP-Hintergrund

Designprinzipien Zusammenfassung

- **Funktionen sind First-Class Citizens**: Grundlage für Callbacks, Higher-Order-Funktionen, funktionale Patterns (`map`/`filter`/`reduce`)
- **Alles, was kein Primitive ist, ist ein Objekt**: Arrays, Funktionen, sogar `null` (historisch fälschlich als „object“ typisiert)
- **Prototypenbasierte Vererbung statt Klassenhierarchie**: Sehr flexibel aber ungewohnt mit typischem OOP-Hintergrund
- **Single-Threaded mit kooperativem Event Loop**: Bewusster Design-Trade-off für Einfachheit im Browser (Vermeidung von Race-Conditions)

JavaScript Syntax

Variablen & Deklaration

- `let` - blockscoped, wiederbeschreibbar
- `const` - blockscoped, nicht neu zuweisbar (Objekte/Arrays bleiben änderbar)
- `var` - veraltet, functionscoped, Hoisting-Probleme, sollte vermieden werden
- Deklaration ohne Zuweisung: `let x;` → Wert: `undefined`

JSON

JSON (JavaScript Object Notation)

- Textbasiertes, sprachunabhängiges Datenformat zum Austausch strukturierter Daten
- Trotz des Namens heute unabhängig von JavaScript
- Nutzbar in jeder Programmiersprache

JSON

```json
{
  "name": "Alex",
  "age": 25,
  "hobbies": ["Lesen", "Programmieren"],
  "address": {
    "city": "Görlitz",
  }
}
```

Erlaubte Datentypen

Typ	Beschreibung
String	immer in doppelten Anführungszeichen, einfache Anführungszeichen sind ungültig
Number	kein Unterschied zwischen Integer und Float
Boolean	`true`/`false`
null	
Object	`{ }`, Key immer als String
Array	`[ ]`

Feste Regeln

- Keys müssen in doppelten Anführungszeichen stehen
- `{name: "x"}` ist **ungültig**
- `{"name": "x"}` ist **gültig**
- Kein Trailing Komma nach dem letzten Eintrag `[1, 2, 3,]` ist ungültig
- Keine Kommentare erlaubt, anders als JavaScript-Objektliterale
- Kein `undefined`, kein `Date`-Typ, keine Funktionen, nur die sechs vorherigen Typen

JSON vs. JavaScript-Objekte

- JavaScript-Objektliteral (`{ name: "Alex" }`) ist Code, keine Daten
- JSON ist immer ein String der geparst wird
- Zwei Methoden in JavaScript

```javascript
const obj = { name: "Alex", age: 25 };
const jsonString = JSON.stringify(obj);
// '{"name": "Alex", "age": 25}'
const parsedObj = JSON.parse(jsonString);
// {name: "Alex", age: 25}
```

- `JSON.stringify()`: JavaScript-Wert → JSON-String (nötig, um Daten z.B. per `fetch` zu versenden)
- `JSON.parse()`: JSON-String → JavaScript-Wert (nötig, um eine empfangene API-Antwort zu nutzen)

JSON und fetch zusammen

JavaScript

```
1   // Senden
2   fetch("/api/users", {
3     method: "POST",
4     headers: { "Content-Type": "application/json" },
5     body: JSON.stringify({ name: "Alex", age: 25 }),
6   });
7
8   // Empfangen
9   const response = await fetch("/api/users/1");
10  const data = await response.json(); // parst die Response
    automatisch
```

Validation

Überblick

Warum validieren?

- Mehr UX als wirkliche Sicherheit
- **Nutzerfreundlichkeit**: Sofortiges Feedback, bevor eine Anfrage überhaupt den Server erreicht, spart Wartezeit und Frustration
- **Datenqualität am Frontend**: Offensichtliche Fehler (leeres Pflichtfeld, falsches E-Mail-Format) frühzeitig abfangen

Warum keine Sicherheitsvorkehrung?

Überblick

Warum validieren?

- Mehr UX als wirkliche Sicherheit
- **Nutzerfreundlichkeit**: Sofortiges Feedback, bevor eine Anfrage überhaupt den Server erreicht, spart Wartezeit und Frustration
- **Datenqualität am Frontend**: Offensichtliche Fehler (leeres Pflichtfeld, falsches E-Mail-Format) frühzeitig abfangen

Warum keine Sicherheitsvorkehrung?

Jede Client-seitige Prüfung lässt sich mit DevTools, deaktiviertem JavaScript, direkten Requests mit `curl`/Postman usw. umgangen werden. Verbindliche Validierung muss immer zusätzlich Server-side geschehen.

Überblick

Merksatz: Client-Validierung ist ein Komfort-Layer, Server-Validierung ist die Wahrheit.

HTML-Validierung

Einfachster Ansatz: Validierung über HTML-Attribute ohne JavaScript

HTML

```
1  <form>
2    <input type="email" required>
3    <input type="number" min="1" max="100">
4    <input type="text" pattern="[A-Za-z]{3,}" minlength="3"
     maxlength="20">
5    <input type="url">
6    <input type="tel" pattern="[0-9]{10,15}">
7    <button type="submit">Absenden</button>
8  </form>
```

HTML-Validierung

```HTML
<input type="email" required>
```

- `required`: Feld darf nicht leer sein

```HTML
<input type="number" min="1" max="100">
```

- `type="number"`: Eingebaute Formatprüfung je nach Typ
- `min`/`max`: Wertebereich bei Zahlen/Datum

```HTML
<input type="text" pattern="[A-Za-z]{3,}"
minlength="3" maxlength="20">
```

- `minlength`/`maxlength`: Textlänge

```HTML
<input type="tel" pattern="[0-9]{10,15}">
```

- `pattern`: Regex für individuelle Formate

HTML-Validierung

Vorteil:

- Browser übernimmt UI (rote Umrandung, Tooltip-Fehlermeldung) und Logik

Nachteil:

- Standard-Fehlermeldung sind nicht anpassbar
- Oft inkonsistent mit dem restlichen Design
- Reicht bei großen Projekten selten aus

Constraint Validation API

Brücke zwischen HTML-Validierung und eigenem JavaScript (selten genutzt aber nützlich)

```javascript
const input = document.querySelector("input[type=email]");

input.checkValidity();
input.validity.valueMissing;
input.validity.typeMismatch;
input.setCustomValidity("Enter a valid email address");
```

- `setCustomValidity` erlaubt eigene Fehlertexte
- Native Browser-UI bleibt erhalten

Eigene JavaScript-Validierung

- Nutzung sobald Regeln hinzukommen, die HTML nicht mehr verarbeiten kann
- Beispiel: Passwort und Passwort-Wiederholung müssen übereinstimmen

JavaScript

```
1   form.addEventListener("submit", (event) => {
2     const password = form.password.value;
3     const confirm = form.passwordConfirm.value;
4
5     if (password != confirm) {
6       event.preventDefault(); // verhindert Absenden
7       showError("Passwörter stimmen nicht überein");
8     }
9   })
```

Zeitpunkt der Validierung

Häufige Validierungszeitpunkte:

- **Bei jedem Tastendruck**: sofortiges Feedback, aber nervig bei z.B.: E-Mail Feldern, die erst nach vollständiger Eingabe gültig sind
- **Beim Verlassen des Feldes**: guter Kompromiss, Standard-Empfehlung für die meisten Formulare
- **Erst beim Absenden**: am wenisten störend, aber Nutzer merken Fehler erst spät

Gängiges Best-Practice-Pattern: Erstvalidierung beim `submit`, danach (sobald ein Fehler sichtbar war) live bei jedem `input`, damit der Nutzer sofort sieht, wann der Fehler behoben ist.

Barrierefreiheit bei Validierungsfehlern

- Fehlermeldungen mit `aria-invalid="true"` am betroffenen Feld markieren
- Fehlertext über `aria-describedBy` mit dem Input verknüpfen, damit Screenreader ihn vorlesen
- Fehler nicht nur über Farbe kommunizieren, sondern auch über Text/Icon

HTML

```
<input
  type="email"
  aria-invalid="true"
  aria-describedby="email-error"
>
<span id="email-error">
  Bitte eine gültige E-Mail-Adresse eingeben
</span>
```

Visuelles Feedback mit CSS

```css
input:invalid {
  border-color: red;
}
input:valid {
  border-color: green;
}
input:focus:invalid {
  outline-color: orange;
}
```

- `:invalid`/`:valid` reagieren automatisch auf die nativen HTML-Validierung
- `:invalid` greift bereits bei leeren `required`-Feldern vor der ersten Interaktion
- Kann über `:placeholder-shown` umgangen werden oder JS

Zusammenfassung

- Native HTML-Attribute zuerst nutzen
- Ist kostenlos und barrierefrei vorbereitet
- Constraint-Validation API für eigene Fehlertexte, bevor man komplett bei null anfängt
- Eigene JavaScript Logik nur für komplexere Regeln
- **Server-seitige Validierung niemals optional**